Listing 12.3 Pact get movies

```
test('it should return an empty list', async () => {
  provider
    .uponReceiving('get movies when none exist in database')
    .withRequest({
      method: 'GET',
      path: '/movies',
    })
    .willRespondWith({
      headers: { 'Content-Type': 'application/json' },
      status: 200,
      body: like([]),
    });

  await provider.executeTest(async mockProvider => {
    const movies = await fetchMovies(mockProvider.url);
    expect(movies).toEqual([]);
  });
});
```

As listing 12.3 demonstrates, the test in listing 12.2 can easily be replicated as a consumer contract test. The benefit of converting this test is improved stability through passing the provider state. Integration environments often come with challenges of keeping the data in the right state between test runs. However, with the provider state in contract tests, the database can be set directly before the test run. Another reason the integration tests may be unstable is because the environment is down or a deployment is happening at the same time as the tests have been triggered. However, contract tests can be run against a local instance of the provider.

As shown in listing 12.4, the app is launched as part of the test file launching the Express server. Within the Pact verifier options, the provider name is referenced, which will pull down all consumer contracts related to the provider. The `uponReceiving` statement within the consumer test is referenced in the `stateHandlers` object as part of the Pact verifier options. The state handler performs actions before the provider tests run, such as deleting all movies within the database. Within the test, the `verifyProvider` function is called, verifying that the local implementation of the API is compatible with the consumer contract.